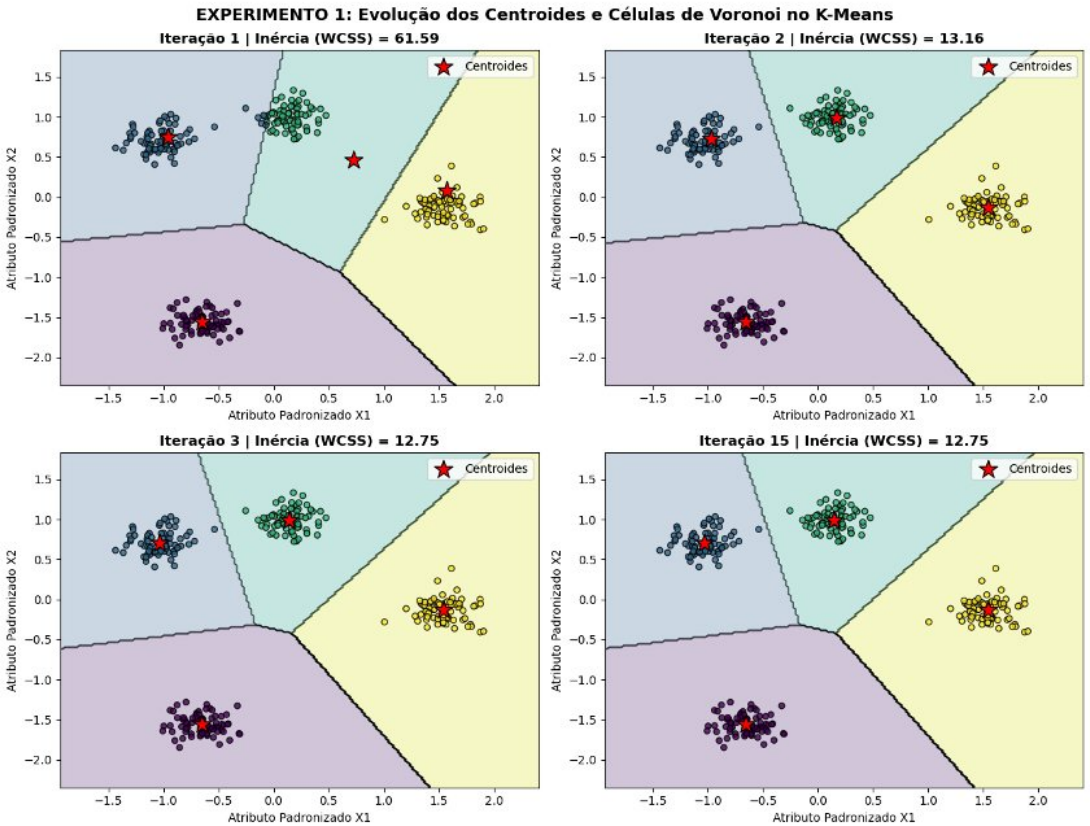
Laboratório Aula 15

[1]:
```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.cm as cm
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans, MiniBatchKMeans
from sklearn.metrics import silhouette_score, silhouette_samples
```

1 Experimento 1: A Dinâmica Iterativa de Lloyd-Forgy e Diagrama de Voronoi

[2]:
```
X_blobs, y_blobs_real = make_blobs(n_samples=300, centers=4, cluster_std=0.85,␣
 ↪random_state=42)
scaler_blobs = StandardScaler()
X_scaled = scaler_blobs.fit_transform(X_blobs)
sementes_iniciais = np.array([[-1.5, -1.5], [-1.0, 1.5], [0.0, 0.0], [1.5, 1.
 ↪5]])
xx, yy = np.meshgrid(np.linspace(X_scaled[:, 0].min() - 0.5, X_scaled[:, 0].
 ↪max() + 0.5, 200),
                     np.linspace(X_scaled[:, 1].min() - 0.5, X_scaled[:, 1].
 ↪max() + 0.5, 200))
grade_voronoi = np.c_[xx.ravel(), yy.ravel()]
passos_iteracao = [1, 2, 3, 15]
fig, eixos = plt.subplots(2, 2, figsize=(13, 10))
for ax, n_iter in zip(eixos.flat, passos_iteracao):
    km_passo = KMeans(n_clusters=4, init=sementes_iniciais, n_init=1,␣
 ↪max_iter=n_iter, random_state=42)
    rotulos_passo = km_passo.fit_predict(X_scaled)
    fundo_voronoi = km_passo.predict(grade_voronoi).reshape(xx.shape)
    ax.contourf(xx, yy, fundo_voronoi, cmap='viridis', alpha=0.25)
    ax.contour(xx, yy, fundo_voronoi, colors='k', linewidths=0.8, alpha=0.5)
    ax.scatter(X_scaled[:, 0], X_scaled[:, 1], c=rotulos_passo, cmap='viridis',␣
 ↪s=25, edgecolor='k', alpha=0.8)
    centros_atuais = km_passo.cluster_centers_
    ax.scatter(centros_atuais[:, 0], centros_atuais[:, 1], c='red', marker='*',␣
 ↪s=260, edgecolor='black', label='Centroides')
```

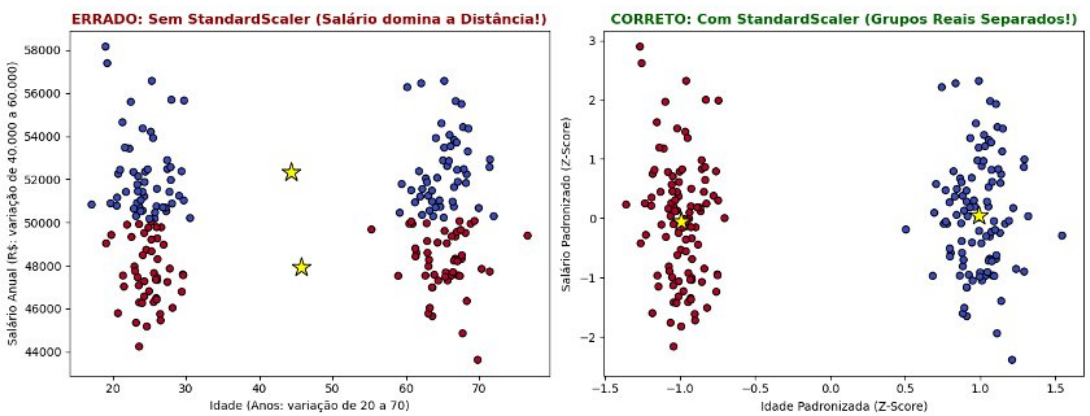
```
    ax.set_title(f"Iteração {n_iter} | Inércia (WCSS) = {km_passo.inertia_:.
  ↪2f}", fontweight='bold')
    ax.set_xlabel("Atributo Padronizado X1")
    ax.set_ylabel("Atributo Padronizado X2")
    ax.legend(loc='upper right')
plt.suptitle("EXPERIMENTO 1: Evolução dos Centroides e Células de Voronoi no␣
  ↪K-Means", fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()
```



2 Experimento 2: O Impacto da Escala na Distância Euclidiana

[3]:
```
np.random.seed(42)
grupo_a = np.column_stack([np.random.normal(25, 3, 100), np.random.
  ↪normal(50000, 3000, 100)])
grupo_b = np.column_stack([np.random.normal(65, 3, 100), np.random.
  ↪normal(50000, 3000, 100)])
X_escala = np.vstack([grupo_a, grupo_b])
km_sem_escala = KMeans(n_clusters=2, n_init=10, random_state=42).fit(X_escala)
```

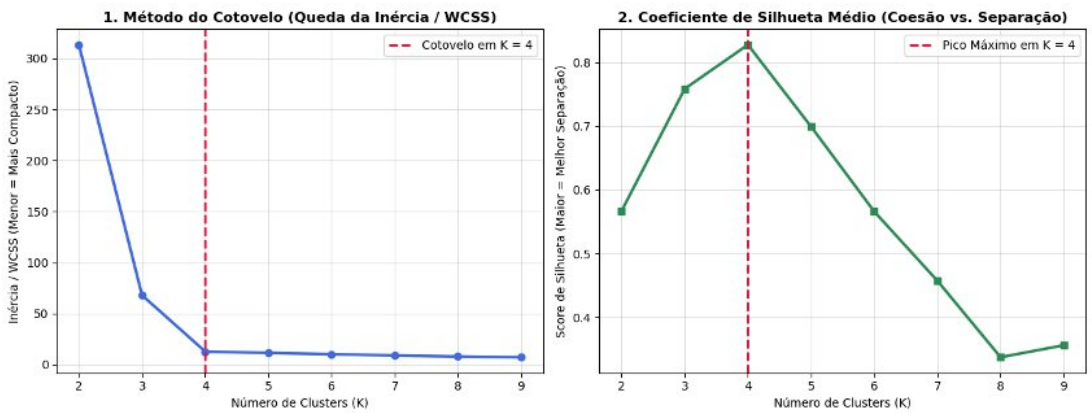
```
X_escala_norm = StandardScaler().fit_transform(X_escala)
km_com_escala = KMeans(n_clusters=2, n_init=10, random_state=42).
 ↪fit(X_escala_norm)
fig, (ax_errado, ax_certo) = plt.subplots(1, 2, figsize=(13, 5))
ax_errado.scatter(X_escala[:, 0], X_escala[:, 1], c=km_sem_escala.labels_,␣
 ↪cmap='coolwarm', s=35, edgecolor='k')
ax_errado.scatter(km_sem_escala.cluster_centers_[:, 0], km_sem_escala.
 ↪cluster_centers_[:, 1], c='yellow', marker='*', s=280, edgecolor='k')
ax_errado.set_title("ERRADO: Sem StandardScaler (Salário domina a Distância!)",␣
 ↪color='darkred', fontweight='bold')
ax_errado.set_xlabel("Idade (Anos: variação de 20 a 70)")
ax_errado.set_ylabel("Salário Anual (R$: variação de 40.000 a 60.000)")
ax_certo.scatter(X_escala_norm[:, 0], X_escala_norm[:, 1], c=km_com_escala.
 ↪labels_, cmap='coolwarm', s=35, edgecolor='k')
ax_certo.scatter(km_com_escala.cluster_centers_[:, 0], km_com_escala.
 ↪cluster_centers_[:, 1], c='yellow', marker='*', s=280, edgecolor='k')
ax_certo.set_title("CORRETO: Com StandardScaler (Grupos Reais Separados!)",␣
 ↪color='darkgreen', fontweight='bold')
ax_certo.set_xlabel("Idade Padronizada (Z-Score)")
ax_certo.set_ylabel("Salário Padronizado (Z-Score)")
plt.tight_layout()
plt.show()
```



3 Experimento 3: Descobrindo o K Ideal – Método do Cotovelo vs. Coeficiente de Silhueta

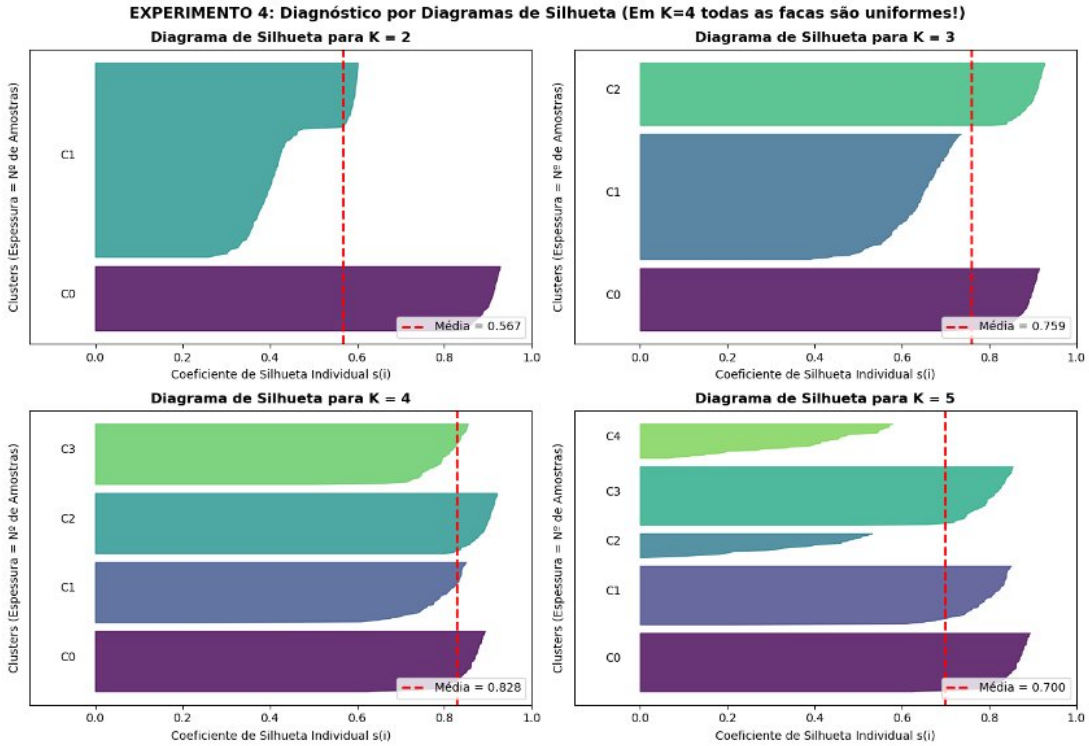
[5]:
```
valores_k = list(range(2, 10))
lista_inercias = []
lista_silhuetas = []
for k in valores_k:
```

```
    km_k = KMeans(n_clusters=k, init='k-means++', n_init=10, random_state=42)
    rotulos_k = km_k.fit_predict(X_scaled)
    lista_inercias.append(km_k.inertia_)
    lista_silhuetas.append(silhouette_score(X_scaled, rotulos_k))
k_otimo = valores_k[int(np.argmax(lista_silhuetas))]
fig, (ax_cot, ax_sil) = plt.subplots(1, 2, figsize=(13, 5))
ax_cot.plot(valores_k, lista_inercias, marker='o', linewidth=2.5,␣
 ↪color='royalblue')
ax_cot.axvline(x=k_otimo, color='crimson', linestyle='--', linewidth=2,␣
 ↪label=f'Cotovelo em K = {k_otimo}')
ax_cot.set_title("1. Método do Cotovelo (Queda da Inércia / WCSS)",␣
 ↪fontweight='bold')
ax_cot.set_xlabel("Número de Clusters (K)")
ax_cot.set_ylabel("Inércia / WCSS (Menor = Mais Compacto)")
ax_cot.grid(True, alpha=0.4)
ax_cot.legend()
ax_sil.plot(valores_k, lista_silhuetas, marker='s', linewidth=2.5,␣
 ↪color='seagreen')
ax_sil.axvline(x=k_otimo, color='crimson', linestyle='--', linewidth=2,␣
 ↪label=f'Pico Máximo em K = {k_otimo}')
ax_sil.set_title("2. Coeficiente de Silhueta Médio (Coesão vs. Separação)",␣
 ↪fontweight='bold')
ax_sil.set_xlabel("Número de Clusters (K)")
ax_sil.set_ylabel("Score de Silhueta (Maior = Melhor Separação)")
ax_sil.grid(True, alpha=0.4)
ax_sil.legend()
plt.tight_layout()
plt.show()
```



4 Experimento 4: Anatomia Interna dos Grupos pelos Diagramas de Silhueta ("Gráfico de Facas")

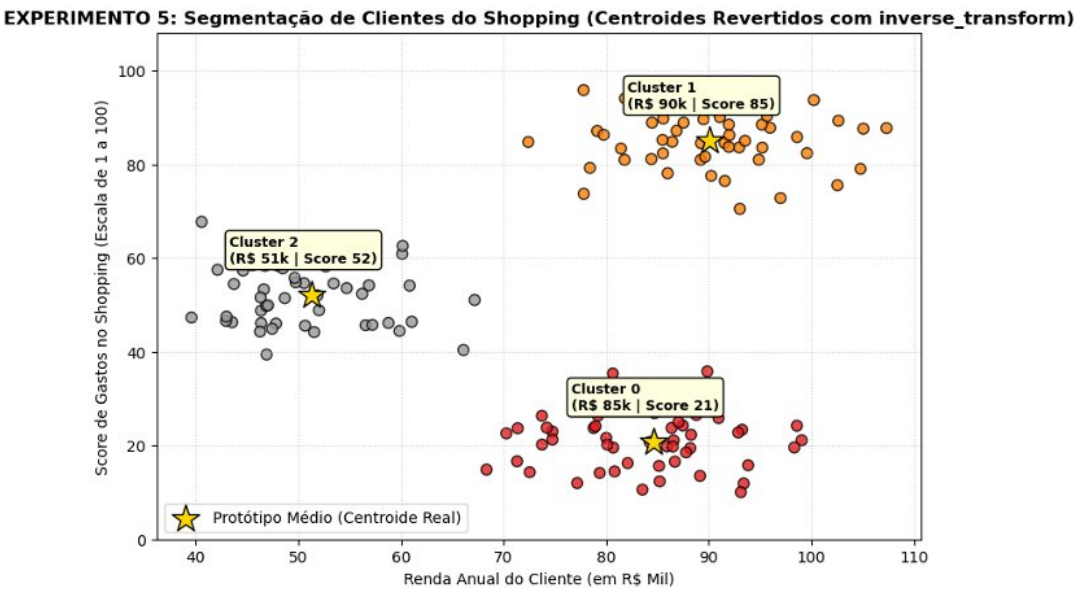
[6]:
```
fig, eixos_faca = plt.subplots(2, 2, figsize=(13, 9))
for ax, k_teste in zip(eixos_faca.flat, [2, 3, 4, 5]):
    km_faca = KMeans(n_clusters=k_teste, init='k-means++', n_init=10,␣
 ↪random_state=42)
    labels_faca = km_faca.fit_predict(X_scaled)
    media_sil = silhouette_score(X_scaled, labels_faca)
    sil_individuais = silhouette_samples(X_scaled, labels_faca)
    y_inferior = 10
    for i in range(k_teste):
        sil_cluster_i = sil_individuais[labels_faca == i]
        sil_cluster_i.sort()
        tamanho_cluster_i = sil_cluster_i.shape[0]
        y_superior = y_inferior + tamanho_cluster_i
        cor = cm.viridis(float(i) / k_teste)
        ax.fill_betweenx(np.arange(y_inferior, y_superior), 0, sil_cluster_i,␣
 ↪facecolor=cor, edgecolor=cor, alpha=0.8)
        ax.text(-0.08, y_inferior + 0.5 * tamanho_cluster_i, f"C{i}")
        y_inferior = y_superior + 10
    ax.axvline(x=media_sil, color="red", linestyle="--", linewidth=2,␣
 ↪label=f"Média = {media_sil:.3f}")
    ax.set_title(f"Diagrama de Silhueta para K = {k_teste}", fontweight='bold')
    ax.set_xlim([-0.15, 1.0])
    ax.set_xlabel("Coeficiente de Silhueta Individual s(i)")
    ax.set_ylabel("Clusters (Espessura = Nº de Amostras)")
    ax.set_yticks([])
    ax.legend(loc="lower right")
plt.suptitle("EXPERIMENTO 4: Diagnóstico por Diagramas de Silhueta (Em K=4␣
 ↪todas as facas são uniformes!)", fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()
```



5 Experimento 5: Segmentação de Clientes com MiniBatchK-Means e inverse_transform

[8]:
```
clientes_economicos = np.column_stack([np.random.normal(85, 8, 50), np.random.
 ↪normal(20, 6, 50)])
clientes_regulares = np.column_stack([np.random.normal(50, 7, 50), np.random.
 ↪normal(50, 7, 50)])
clientes_vip = np.column_stack([np.random.normal(90, 8, 50), np.random.
 ↪normal(85, 6, 50)])
X_shopping = np.vstack([clientes_economicos, clientes_regulares, clientes_vip])
scaler_shop = StandardScaler()
X_shop_scaled = scaler_shop.fit_transform(X_shopping)
mb_shop = MiniBatchKMeans(n_clusters=3, batch_size=32, n_init=10,␣
 ↪random_state=42)
clusters_shop = mb_shop.fit_predict(X_shop_scaled)
centroides_reais_shop = scaler_shop.inverse_transform(mb_shop.cluster_centers_)
plt.figure(figsize=(9, 6))
plt.scatter(X_shopping[:, 0], X_shopping[:, 1], c=clusters_shop, cmap='Set1',␣
 ↪s=50, edgecolor='k', alpha=0.8)
```

```
plt.scatter(centroides_reais_shop[:, 0], centroides_reais_shop[:, 1], c='gold',␣
 ↪marker='*', s=350, edgecolor='black', label='Protótipo Médio (Centroide␣
 ↪Real)')
for idx, (renda_m, score_m) in enumerate(centroides_reais_shop):
    plt.annotate(f"Cluster {idx}\n(R$ {renda_m:.0f}k | Score {score_m:.0f})",
                 xy=(renda_m, score_m),
                 xytext=(renda_m - 8, score_m + 7),
                 fontsize=9, fontweight='bold',
                 bbox=dict(boxstyle="round,pad=0.3", fc="lightyellow",␣
 ↪ec="black", lw=1))
plt.title("EXPERIMENTO 5: Segmentação de Clientes do Shopping (Centroides␣
 ↪Revertidos com inverse_transform)", fontweight='bold')
plt.xlabel("Renda Anual do Cliente (em R$ Mil)")
plt.ylabel("Score de Gastos no Shopping (Escala de 1 a 100)")
plt.ylim(0, 108)
plt.grid(True, linestyle=':', alpha=0.6)
plt.legend(loc='lower left')
plt.show()
```



[ ]: